

Assignment 6 - DDL: Database Modeling

Team: MEB

Customer: Barak Blaish

Designer: Michel Weingarten

Reviewer: Eran Golani

1. Main Use Case (written by customer)

ChargePoint Israel operates EV charging stations across the city. Customers sign up via a mobile app, find a nearby station, pick a free port and start charging. The system tracks energy used and charges the customer automatically when the session ends. The operations team monitors port status, handles maintenance, and ensures payments are correct. Customer service needs fast access to session and payment history for disputes.

Customer Requirements

R1 - Customers: Store name, email, phone, date of birth. Each customer has a unique ID.

R2 - Stations and ports: Track stations by location. Each station has ports with different connector types and power levels. Station address stored once, not repeated per port.

R3 - Port availability: Track the current status of each port. Unavailable ports cannot be assigned to new sessions.

R4 - Sessions: Record every session with customer, port, start and end time, and total energy in kWh.

R5 - Payments: Record a payment per session with payment status to support dispute handling.

R6 - Incidents: Record fault incidents linked to a port and optionally to a session so we can trace responsibility.

R7 - Maintenance: Record maintenance events per port with type, description, and dates.

Note: After discussion with the designer, R2 was updated to separate station and port data into two tables. After reviewer feedback, R5 was updated to include payment status tracking.

2. Designer's Understanding (written by designer)

The main challenge is tracking the real-time status of every port individually. Each port has its own connector type and power level, so we cannot just count free ports per station. Port availability is stored as a status field, not calculated from session overlaps, because a port can go offline for reasons unrelated to active sessions. Incidents and maintenance are stored in separate tables because incidents may be linked to a customer session and could lead to billing disputes, while maintenance is a planned internal activity. Every completed session creates a payment record with a status field to handle failed or disputed payments.

3. Design Discussions (written by designer)

Discussion 1 - Track each port individually or per station

Customer: I thought we could count free ports per station since they share the same location.

Designer: Each port has its own connector type and power level, so individual tracking is needed. Station data goes in a separate table to avoid repeating the address in every port row.

Reviewer: Agreed. Separating stations from ports is the right normalization choice.

Final Decision:

charging_ports table with foreign key to stations table.

Discussion 2 – Port availability tracking

Customer: Should we check availability from overlapping session times or store a status field?

Designer: Status field is better. A port can go offline for reasons outside active sessions. Checking overlapping sessions would be slow and miss these cases.

Reviewer: Agreed. Status values must be clearly defined. This is a deliberate choice — availability could be calculated but storing it directly is faster for daily operations.

Final Decision:

charging_ports.status with values: available, in_use, maintenance, out_of_order, retired.

Discussion 3 - One issues table or separate tables

Customer: I suggested one table for both maintenance and incidents since both take a port offline.

Designer: Maintenance is planned by staff. Incidents are unexpected and may be linked to a session. Mixing them creates empty columns and messy queries.

Reviewer: Agreed. incidents.session_id should be nullable since some faults happen after a session ends.

Final Decision:

Two tables: maintenance_records linked to port only. incidents linked to port and optionally to session.

Discussion 4 - Payment status and disputes

Customer: We need to handle failed payments and customer disputes.

Designer: Added payment_status column to payments table with values: pending, completed, failed, refunded.

Reviewer: Approved. Status updates should be handled by the application, not entered manually.

Final Decision:

payments table includes payment_status column.

4. Reviewer Summary (written by reviewer)

I verified the design fits all requirements and follows normalization. The design is organized and ready for daily operations.

Key Improvements Identified During Review:

1. Separated station data from port data into two tables to avoid address duplication.
2. Added session_id as nullable FK in incidents to trace faults back to the responsible customer.
3. Added payment_status to the payments table for failed payments and disputes.
4. Confirmed that charging_ports.status is a deliberate design choice for performance. The application keeps it updated.
5. Recommended UNIQUE constraint on email in customers table.

Final Agreed System Design

Seven tables connected by primary and foreign keys: customers, stations, charging_ports, sessions, payments, maintenance_records, incidents.

How the design fits the requirements:

1. customers - covers R1
2. stations + charging_ports - covers R2
3. charging_ports.status - covers R3
4. sessions - covers R4
5. payments + payment_status - covers R5
6. incidents - covers R6
7. maintenance_records - covers R7

